



YANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Alternate Assessment Tool

Course Title: INTERNET OF THINGS AND 5G

Course Code: 22AI642

Semester: VI

Submitted by

Sathwik Kumar Shetty – 1DS23AI050

Under the Guidance of

Dr Aruna M G

Professor

Department of Artificial Intelligence & Machine Learning

DSCE, Bengaluru

Visvesvaraya Technological University

Jnanasangama, Belagavi-590018, Karnataka, India

2025-26

Table of Contents

Contents

Activity 1: TinkerCAD Aurdino	1
Activity 2: Wokwi ESP32	8
Activity 3: Wokwi STM.....	15
Activity 4: Wokwi Raspberry Pi Pico simulator.....	23
Activity 5 : TinyML simulation in Wokwi	29

Activity 1: TinkerCAD Aurdino

Problem Statement	The lack of real-time monitoring in solar power systems leads to inefficient energy utilization, undetected faults, and reduced system performance. Users often rely on manual inspection, making it difficult to track power generation, detect anomalies, and optimize energy usage.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	TinkerCAD, Aurdino, Solar Panel, LCD Display				
MAP SDG	7.1: Ensure access to affordable, reliable, and modern energy services. 7.2: Increase the share of renewable energy in the global energy mix. 7.3: Improve energy efficiency through monitoring and optimization. 11.6: Reduce environmental impact by promoting clean energy adoption.				
MAP COURSE OUTCOME of IOT	CO	Mapping	Justification		
	CO1	Yes	Demonstrates the perception layer of IoT architecture where voltage data from the solar panel is sensed, processed by the microcontroller, and displayed via LCD/Serial output.		
	CO2	Yes	Uses Arduino C programming, analog-to-digital conversion for voltage measurement, and I2C/Serial communication protocols for data display and transmission.		
	CO3	No	Basic data observation is performed, but advanced data analytics is not implemented in this standalone monitoring system.		
	CO4	Yes	Utilizes voltage sensing as input and display units such as an LCD or Serial Monitor for output, forming a complete sense–process–display IoT loop, where sensor data is acquired, processed using embedded logic or machine learning, and presented to the user in real time for effective monitoring and decision-making.		

MAP PROGRAM OUTCOME	PO	Mapping	Justification
	PO1	Yes	Applied knowledge of electronics, mathematics (voltage and power calculations), and computing fundamentals.
	PO2	Yes	Identified and analyzed the real-world problem of inefficient solar energy utilization due to lack of monitoring and performance tracking.
	PO3	Yes	Designed a system to monitor solar voltage and estimate power output, enabling better energy management and performance awareness.
	PO5	Yes	Used Tinkercad (modern engineering tools) for simulation and Arduino IDE for programming and implementation.
	PO6	Yes	Contributes to societal and environmental sustainability by promoting efficient use of renewable energy and reducing dependency on non-renewable sources.
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO 1	Yes	Demonstrates programming and problem-solving using Arduino C code for sensor data acquisition, voltage calculation, and real-time display of results.
	PSO 2	Yes	Identifies and addresses the real-world problem of monitoring solar energy systems using embedded systems and sensing techniques.
	PSO 3	Yes	Uses Tinkercad as computational platform to deploy and test the IoT solution.
URL Link	https://www.tinkercad.com/things/aWQHtHJFLtJ-sloar-power-monitoring?sharecode=ZO54QH4rhB79_Yzip1PTysB4sZ3AhKkEJfWXNDA6rs		

SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	<pre>graph TD; SolarMonitor -- uses --> VoltageSensor; SolarMonitor -- uses --> LCDDisplay; SolarMonitor -- uses --> SerialMonitor;</pre> SolarMonitor - sampleInterval : int - threshold : float - status : String + setup() : void + loop() : void VoltageSensor - analogPin : int - scaleFactor : float - rawValue : int + readVoltage() : float + readPower() : float + simulate() : float LCDDisplay - pin : int - cols : int - rows : int + begin() : void + displayVoltage() : void + displayPower() : void + clear() : void SerialMonitor - baudRate : int - lastOutput : String + printStatus() : void + logData() : void + begin() : void
COURSE COORDINATOR	Dr ARUNA M G

Details
Abstract : The increasing adoption of solar energy systems necessitates efficient monitoring to ensure optimal performance and reliability. This project presents a real-time solar power monitoring system that tracks key parameters such as voltage, current, and power output. The system enables continuous observation, helping users detect faults, inefficiencies, and performance drops without manual intervention. By providing actionable insights, it enhances energy utilization and supports preventive maintenance. Overall, the solution contributes to smarter renewable energy management and promotes sustainable energy practices.
Use Case: The system can be used by households and industries to monitor solar panel performance and optimize energy consumption in real time. It helps maintenance teams detect faults early, reducing downtime and improving system reliability. Additionally, it supports smart energy management in institutions and smart cities by enabling data-driven decisions for efficient renewable energy usage.
Introduction: The increasing demand for clean and sustainable energy has accelerated the adoption of solar power systems. However, the lack of real-time monitoring often leads to inefficiencies and undetected faults. Traditional methods rely on manual inspection, which is time-consuming and unreliable. This project introduces a solar power monitoring system that enables continuous tracking and improves overall system performance.
Objective: • To design a system for real-time monitoring of solar power parameters such as voltage, current, and power output. • To detect faults and performance issues early, enabling timely maintenance and reducing downtime.

- To improve overall energy efficiency through continuous data analysis and insights.
- To provide a user-friendly interface for easy visualization and management of solar system performance.

General procedure

1. Connect the solar panel and sensors (voltage/current) to the microcontroller.
2. Provide power supply to the entire monitoring circuit.
3. Upload the program (code) to the microcontroller.
4. Start the system and begin data acquisition.
5. Observe and analyze the power parameters on the monitoring interface.

Step to Execute:

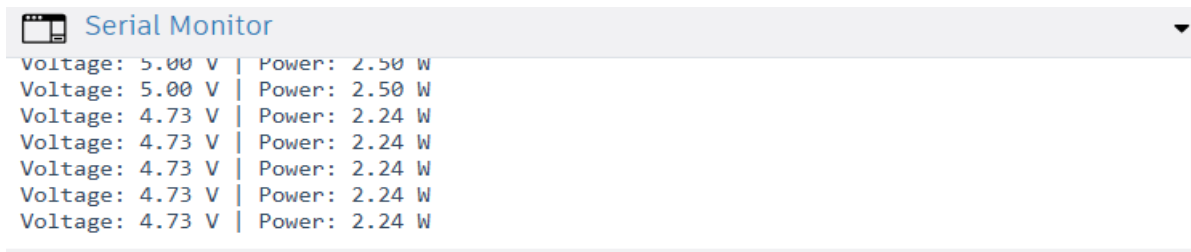
1. Open the simulation platform (Tinkercad).
2. Place components: Arduino UNO, Solar Panel, 16x2 LCD Display, Breadboard, Resistors.
3. Connect the solar panel output to the breadboard circuit for voltage sensing.
4. Interface the LCD display with Arduino.
5. Write or paste the Arduino code to read voltage and display it on the LCD.
6. Click on “Start Simulation”.
7. Observe real-time voltage/power values on the LCD display.
8. Vary light intensity (simulation control) to analyze system performance.

Source Code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
void setup() {
  Wire.begin();
  lcd.init();
  lcd.backlight();
```

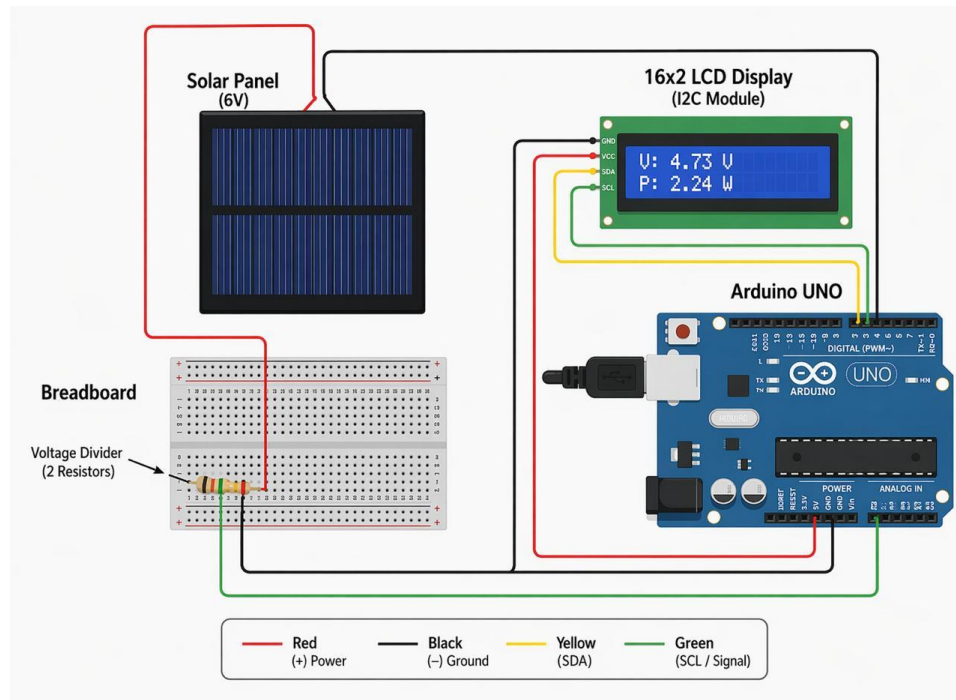
```
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("Solar Monitor");  
delay(2000);  
lcd.clear();  
}  
void loop() {  
  int rawValue = analogRead(A0);  
  float voltage = rawValue * (5.0 / 1023.0) * 4.0;  
  float power = (voltage * voltage) / 10.0;  
  lcd.setCursor(0, 0);  
  lcd.print("V: ");  
  lcd.print(voltage, 2);  
  lcd.setCursor(0, 1);  
  lcd.print("P: ");  
  lcd.print(power, 2);  
  lcd.print(" W  ");  
  delay(1000);  
}
```

Output:



```
Serial Monitor  
Voltage: 5.00 V | Power: 2.50 W  
Voltage: 5.00 V | Power: 2.50 W  
Voltage: 4.73 V | Power: 2.24 W  
Voltage: 4.73 V | Power: 2.24 W  
Voltage: 4.73 V | Power: 2.24 W  
Voltage: 4.73 V | Power: 2.24 W  
Voltage: 4.73 V | Power: 2.24 W
```


Photos/ Screenshot/Snapshot :



Outcome:

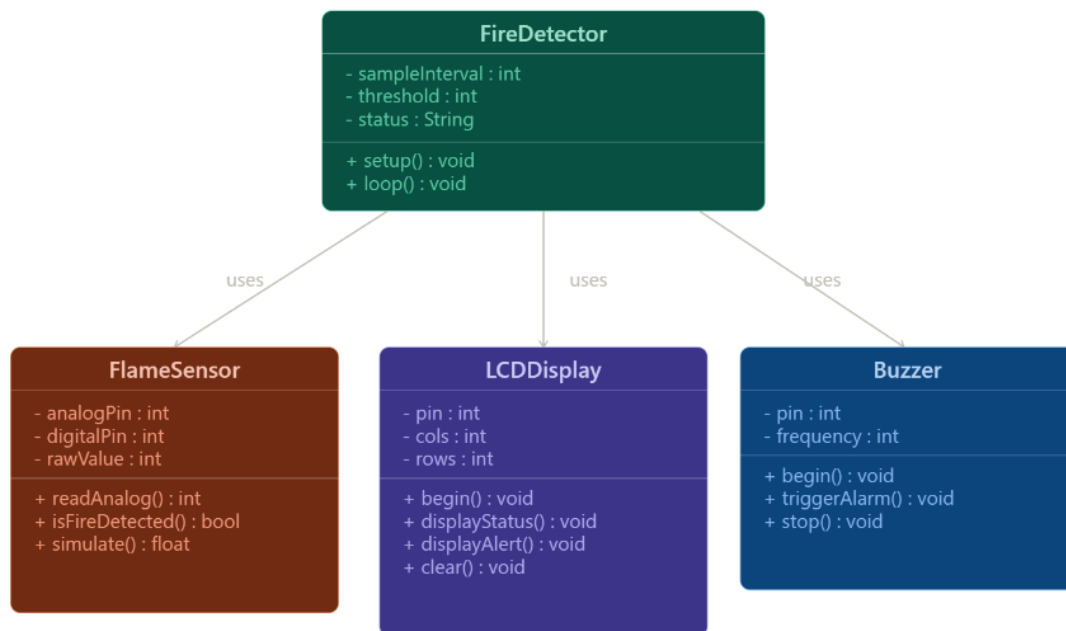
The system successfully monitors solar voltage and calculates power output in real time. It displays the data on both an LCD screen and Serial Monitor for easy visualization and analysis. The project improves efficiency by enabling continuous monitoring and early detection of performance variations.

Activity 2: Wokwi ESP32

Problem Statement	Unattended or abnormal conditions such as high temperature and gas leakage can lead to fire hazards in homes and industries. Traditional systems rely on fixed thresholds, which may fail to detect complex or gradual anomalies. Delayed detection of such conditions can result in severe damage to property and risk to human life. There is a need for an intelligent system that can continuously monitor sensor data and identify potential fire risks in real time. A TinyML-based approach can enable accurate, fast, and energy-efficient detection directly on edge devices.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	Wokwi, ESP32, DHT22, Piezo Buzzer, Red Led				
MAP SDG	3.2: Reduce preventable deaths through early detection of hazards like fire. 3.9: Reduce illnesses and deaths from hazardous situations and environmental risks. 11.5: Reduce the impact of disasters, including fire-related damages.				
MAP COURSE OUTCOME of IOT/ BFE	CO	Mapping	Justification		
	CO1	Yes	Demonstrates the perception layer of IoT architecture where the DHT22 sensor collects temperature data, the ESP32 processes it, and alerts are generated through LED and buzzer (application layer).		
	CO2	Yes	Uses ESP32 programming with Arduino, and implements sensor interfacing protocols (GPIO-based communication with DHT22) along with Serial communication for monitoring.		
	CO3	Yes	Performs real-time data monitoring through Serial output, though advanced cloud-based analytics is not implemented in this standalone system.		

	CO4	Yes	Integrates sensing (DHT22 temperature sensor) with actuation (buzzer and LED) to create a complete sense-process-act embedded system loop.
MAP PROGRAM OUTCOME	PO	Mapping	Justification
	PO1	Yes	Applied knowledge of temperature sensing, embedded systems, and basic electronics for fire detection using the DHT22 sensor.
	PO3	Yes	Designed a real-time fire detection system that identifies high-temperature conditions and alerts users for safety.
	PO5	Yes	Used Wokwi simulator and ESP32 platform along with sensor interfacing and Serial monitoring as modern engineering tools.
	PO9	Yes	Demonstrates communication through Serial output and alert mechanisms (LED and buzzer) to effectively convey fire hazard information.
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO 1	Yes	Demonstrates ESP32 programming with sensor integration (DHT22), implementing temperature-based logic and alert mechanisms using buzzer and LED.
	PSO 2	Yes	Addresses the real-world problem of fire hazard detection using embedded systems and environmental sensing techniques.
	PSO 3	Yes	Utilizes the Wokwi platform to simulate, test, and validate the fire detection system effectively, enabling accurate analysis of system behavior, real-time debugging, and reliable performance verification before deployment on physical hardware.
URL Link	https://wokwi.com/projects/461125002532742145		

**SYSTEM
DIAGRAM / UML
/ SEQUENCE
DIGRAM/ 3D
Model**



**COURSE
COORDINATOR**

Dr ARUNA M G

Details
Abstract : The fire detection system is designed to identify the presence of fire or smoke at an early stage. It helps in reducing damage by providing timely alerts to users. The system is simple, reliable, and suitable for safety applications in homes and public spaces.
Use Case: The fire detection system can be used in homes, offices, and public places to ensure safety from fire hazards. It helps in detecting fire at an early stage and provides timely alerts to users. This enables quick action, reducing damage to life and property.
Introduction: Fire accidents are a major safety concern that can lead to serious damage and loss of life. A fire detection system helps in identifying fire or smoke at an early stage. This project provides a simple and effective solution to enhance safety and prevent disasters.
Objective: • To detect the presence of fire or smoke at an early stage. • To alert users immediately when a fire is detected. • To reduce damage to life and property. • To provide a simple, reliable, and cost-effective safety system.
General procedure 1. Connect the flame sensor to the Arduino along with LED and buzzer as shown in the circuit. 2. Power the system and initialize the pins for input (sensor) and output (LED, buzzer). 3. Continuously read the flame sensor values to check for fire presence. 4. When flame is detected (LOW signal), activate the buzzer and LED as an alert. 5. Display the fire detection status on the serial monitor for monitoring.
Step to Execute: 1. Open the Wokwi ESP32 simulator.

2. Add components: ESP32, DHT22 temperature sensor, LED, and buzzer.
3. Connect VCC (DHT22) to 3.3V of ESP32 and GND to GND.
4. Connect the DATA pin of DHT22 to a GPIO pin (as shown in your circuit).
5. Connect the LED to a GPIO pin through a resistor and GND.
6. Connect the buzzer to a GPIO pin and GND.
7. Paste the ESP32 code in the editor.
8. Click on “Start Simulation”.
9. Adjust the temperature value in DHT22 (simulate fire condition).
10. Observe LED and buzzer activation when temperature exceeds threshold and check Serial Monitor output.

Source Code

```
from machine import Pin
import dht
import time
dht_sensor = dht.DHT22(Pin(17))
buzzer_pin = Pin(15, Pin.OUT)
led = Pin(2, Pin.OUT)
TEMP_THRESHOLD = 30
def buzz(frequency, duration):
    period = 1 / frequency
    cycles = int(frequency * duration)
    for _ in range(cycles):
        buzzer_pin.on()
        time.sleep(period / 2)
        buzzer_pin.off()
        time.sleep(period / 2)
while True:
    try:
        dht_sensor.measure()
        temperature = dht_sensor.temperature()
        humidity = dht_sensor.humidity()
```

```

print(f'Temperature: {temperature}°C, Humidity: {humidity}%')
if temperature > TEMP_THRESHOLD:
    buzz(1000, 0.5)
    led.on()
    time.sleep(0.2)
    led.off()
    time.sleep(0.2)
    print("RUNNN SAVE YOUR LIFE!!")
else:
    led.off()
except Exception as e:
    print("Error reading from sensor:", e)
time.sleep(2)

```

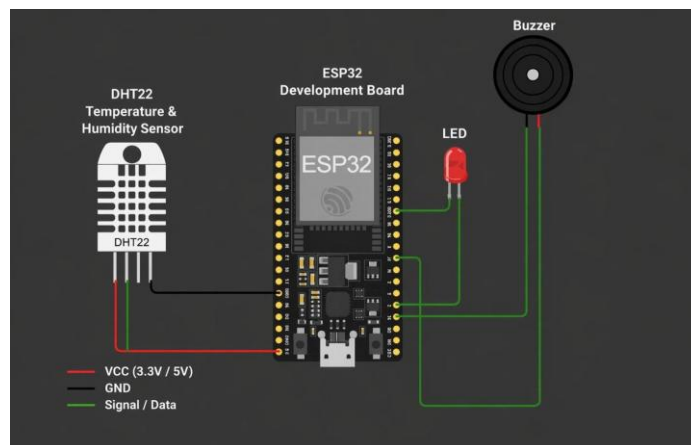
Output:

```

-----
Temperature: 24.0°C, Humidity: 40.0%
Temperature: 58.8°C, Humidity: 40.0%
RUNNN SAVE YOUR LIFE!!
Temperature: 58.8°C, Humidity: 40.0%
RUNNN SAVE YOUR LIFE!!
Temperature: 24.0°C, Humidity: 40.0%
Temperature: 58.8°C, Humidity: 40.0%
RUNNN SAVE YOUR LIFE!!
Temperature: 58.8°C, Humidity: 40.0%
RUNNN SAVE YOUR LIFE!!
^

```

Photos/ Screenshot/Snapshot :



Outcome:

The system simulates temperature and humidity readings for fire detection. It identifies high temperature conditions and indicates a potential fire hazard. Clear alert messages are displayed on the serial monitor for easy monitoring.

Activity 3: Wokwi STM

Problem Statement	The increasing incidents of theft and unauthorized intrusion highlight the growing need for reliable and affordable security systems. Many traditional security solutions are expensive and not easily accessible to common users. Additionally, they often lack real-time detection and immediate alert mechanisms, reducing their effectiveness in preventing incidents. This creates a demand for a cost-effective and efficient system that can continuously monitor surroundings. Therefore, an intelligent burglar detection system is required to provide timely alerts, enhance security, and improve overall safety.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	Wokwi STM, Resistors, Piezo Buzzer, Led, UltraSonic Sensor				
MAP SDG	• 11.5: Reduce the impact of disasters, including crime and security threats. • 11.7: Provide safe and secure public and residential spaces. • 16.1: Significantly reduce all forms of violence and related crime. • 9.1: Develop reliable and resilient infrastructure. • 9.5: Enhance technological capabilities for innovative solutions.				
MAP COURSE	CO	Mapping	Justification		
OUTCOME of IOT/ BFE	CO1	Yes	Demonstrates the perception layer where the ultrasonic sensor measures distance and feeds data to the microcontroller for processing.		
	CO2	Yes	Uses STM32 Arduino programming with digital GPIO interfacing and Serial communication for real-time output.		
	CO3	No	Data analytics is not applicable as this is a standalone real-time detection system.		
	CO4	Yes	Integrates sensing (ultrasonic sensor) with actuation (buzzer and LED) to form a complete sense–process–act embedded		

			system loop.
MAP PROGRAM OUTCOME	PO	Mapping	Justification
	PO1	Yes	Applied knowledge of electronics and embedded systems for distance measurement using ultrasonic sensing and real-time processing.
	PO2	Yes	Formulated the problem of intrusion detection as a distance-based threshold classification using sensor data.
	PO4	Yes	Analyzed sensor readings and implemented threshold-based decision logic to accurately detect potential intrusion events.
	PO5	Yes	Used Wokwi STM32/ESP32 simulator and microcontroller GPIO interfaces as modern engineering tools for system development and testing.
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO 1	Yes	Demonstrates embedded C programming by integrating an ultrasonic sensor with real-time intrusion detection logic, enabling accurate distance measurement and immediate response on the microcontroller.
	PSO 2	Yes	Addresses the real-world problem of theft detection by using distance-based sensing techniques combined with alert mechanisms, providing an effective and practical security solution.
	PSO 3	Yes	Deployed and validated using the Wokwi STM32 simulation platform, enabling accurate testing, debugging, and verification of system functionality.

URL Link	https://wokwi.com/projects/461107253426728961
SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	<pre> classDiagram class BurglarDetector { - sampleInterval : int - alertThreshold : int - status : String + setup() : void + loop() : void } class PIRSensor { - digitalPin : int - sensitivity : float - lastState : int + readMotion() : bool + isTriggered() : bool + reset() : void } class LCDDisplay { - pin : int - cols : int - rows : int + begin() : void + displayStatus() : void + displayAlert() : void + clear() : void } class Buzzer { - pin : int - frequency : int + begin() : void + triggerAlarm() : void + stop() : void } BurglarDetector --> PIRSensor : uses BurglarDetector --> LCDDisplay : uses BurglarDetector --> Buzzer : uses </pre> The diagram shows a BurglarDetector class at the top, which uses three other classes: PIRSensor, LCDDisplay, and Buzzer. Each class has its own set of attributes and methods.
COURSE COORDINATOR	Dr ARUNA M G

Details
Abstract : With the rise in theft and unauthorized access, there is a growing need for simple and effective security solutions. This project presents a burglar detection system using sensors and a microcontroller to identify intrusion based on proximity or movement. When an object is detected within a predefined range, the system triggers an alert using a buzzer and displays status information. The system provides real-time monitoring and immediate response, improving safety and awareness. Overall, it offers a cost-effective and reliable approach to basic security automation.
Use Case: The system can be used in homes to detect unauthorized entry and provide immediate alerts. It is suitable for offices and shops to enhance security during non-working hours. The system can be implemented in warehouses to monitor restricted areas and prevent theft. It can also be used in smart security systems for real-time intrusion detection and automation.
Introduction: With the increasing need for safety and security, burglar detection systems have become essential in modern environments. Traditional security methods are often expensive and lack real-time responsiveness. This project introduces a simple and cost-effective system that detects intrusion using sensors and a microcontroller. It provides immediate alerts, helping to enhance security and prevent unauthorized access.
Objective: • To design a burglar detection system using sensors and a microcontroller. • To detect intrusion based on distance or motion and trigger an alert. • To provide real-time notification using a buzzer and display system. • To develop a cost-effective and reliable solution for basic security needs.
General procedure 1. Connect the ESP32, ultrasonic sensor (HC-SR04), and buzzer as per the circuit design.

-
2. Provide power to the ESP32 using USB and ensure proper VCC and GND connections.
 3. Upload the program to the ESP32 using the Arduino IDE.
 4. Continuously measure distance using the ultrasonic sensor connected to ESP32 GPIO pins.
 5. Activate the buzzer when the detected distance is below the predefined threshold, indicating intrusion.

Step to Execute:

1. Open the Wokwi ESP32 simulator.
2. Add components: ESP32, Ultrasonic Sensor (HC-SR04), Buzzer (and LCD if used). Connect VCC, GND, TRIG, and ECHO pins of the ultrasonic sensor to ESP32 GPIO pins.
3. Connect the buzzer to a GPIO pin and GND.
4. Paste the ESP32 code in the editor.
5. Click on “Start Simulation”.
6. Vary the distance using the ultrasonic sensor controls.
7. Observe buzzer activation and output in Serial Monitor / display.

Source Code:

```
#define TRIG_PIN PA0
#define ECHO_PIN PA1
#define BUZZER_PIN PA2
#define LED_PIN PA3
#define THRESHOLD_CM 50

void setup() {
  Serial.begin(115200);
  while (!Serial);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, HIGH);
}
```

```
digitalWrite(BUZZER_PIN, HIGH);
delay(300);
digitalWrite(LED_PIN, LOW);
digitalWrite(BUZZER_PIN, LOW);
}

float getDistance() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duration = pulseIn(ECHO_PIN, HIGH, 30000);
    if (duration == 0) return -1;
    float distance = (duration * 0.0343) / 2.0;
    return distance;
}

void triggerAlarm() {
    for (int I = 0; I < 3; i++) {
        digitalWrite(BUZZER_PIN, HIGH);
        digitalWrite(LED_PIN, HIGH);
        delay(100);
        digitalWrite(BUZZER_PIN, LOW);
        digitalWrite(LED_PIN, LOW);
        delay(100);
    }
}

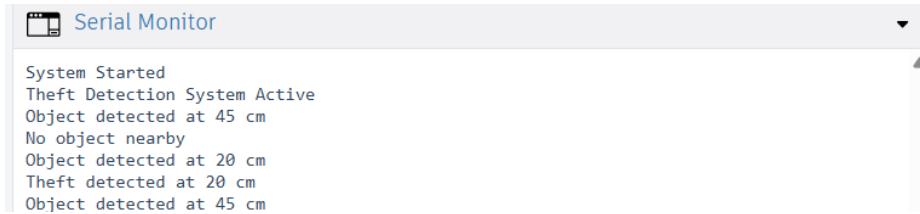
void loop() {
    float distance = getDistance();
    if (distance == -1) {
        Serial.println("[WARN] No echo — sensor out of range or disconnected");
        digitalWrite(BUZZER_PIN, LOW);
        digitalWrite(LED_PIN, LOW);
    }
}
```

```

} else {
  Serial.print("[INFO] Distance: ");
  Serial.print(distance, 2);
  Serial.println(" cm");
  if (distance < THRESHOLD_CM) {
    Serial.println("[ALERT] Object detected! ALARM ON");
    triggerAlarm();
  } else {
    Serial.println("[OK] All clear");
    digitalWrite(BUZZER_PIN, LOW);
    digitalWrite(LED_PIN, LOW);
  } }
delay(200);
}

```

Output:

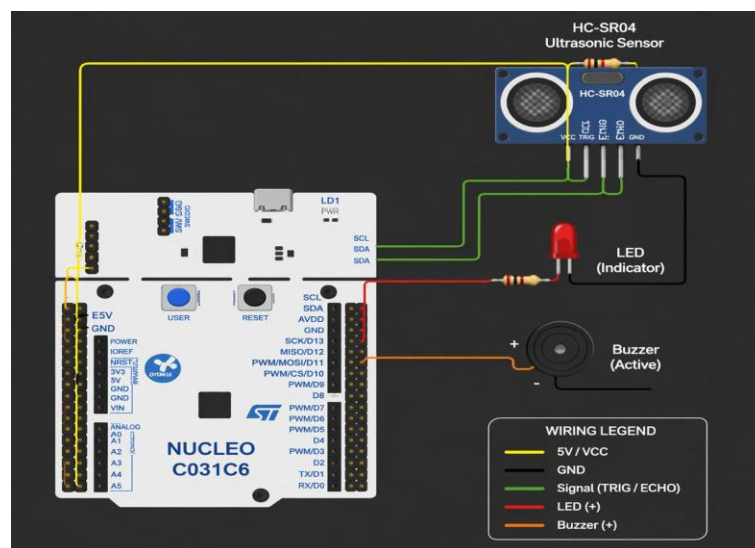


```

Serial Monitor
System Started
Theft Detection System Active
Object detected at 45 cm
No object nearby
Object detected at 20 cm
Theft detected at 20 cm
Object detected at 45 cm

```

Photos/ Screenshot/Snapshot :



Outcome:

The system simulates object detection and identifies potential theft when an object is within a set range. It displays simple status messages like safe or theft detected. All outputs are shown on the serial monitor for easy demonstration.

Activity 4: Wokwi Raspberry Pi Pico simulator

Problem Statement	Gas leaks can lead to dangerous situations such as fires, explosions, and serious health hazards if not detected early. Traditional detection methods may not always provide timely or accurate alerts, increasing the risk of accidents. Therefore, there is a need for an intelligent system that can continuously monitor the environment and quickly detect the presence of harmful gases. An efficient gas detection system can provide early warnings, enabling preventive actions to be taken in time. This helps in reducing potential damage, improving safety, and ensuring reliable monitoring in both domestic and industrial settings.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	Wokwi, MQ gas sensor, Raspberry Pi Pico, Piezo Buzzer				
MAP SDG	3.9: Reduce illnesses and deaths caused by hazardous gases and pollution. 11.5: Reduce impact of disasters such as gas leaks and explosions.				
MAP COURSE	CO	Mapping	Justification		
OUTCOME of IOT/ BFE	CO1	Yes	Demonstrates the perception layer using the MQ gas sensor for detecting gas levels, with processing by the Raspberry Pi Pico and alert output via buzzer/LED (application layer).		
	CO2	Yes	Uses MicroPython programming with ADC-based gas sensing and GPIO control, along with Serial output for monitoring.		
	CO3	Yes	Data analytics is not implemented; the system performs real-time gas detection and alerting.		
	CO4	Yes	Integrates sensing (MQ gas sensor) with actuation (buzzer and alert mechanisms) to form a complete sense process act embedded system loop.		

MAP PROGRAM OUTCOME	PO	Mapping	Justification
	PO1	Yes	Applied knowledge of electronics (ADC conversion for gas sensing), timing logic, and embedded communication principles for system operation.
	PO3	Yes	Designed a safety system that detects hazardous gas levels using sensor inputs and triggers timely alerts through actuators, enabling early warning and helping prevent potential accidents, property damage, and risks to human life.
	PO5	Yes	Used Wokwi Raspberry Pi Pico simulator and MicroPython as modern engineering tools for development and testing.
	PO6	Yes	Contributes to public safety by preventing accidents such as gas leaks, fires, and health hazards through early detection and alert mechanisms.
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO 1	Yes	Demonstrates MicroPython programming by integrating an ADC-based gas sensor with real-time alert control using a buzzer and actuators, enabling efficient monitoring and rapid response to hazardous conditions.
	PSO 2	Yes	Addresses the real-world problem of gas leakage and fire hazards using environmental sensing and embedded system design.
	PSO 3	Yes	Deployed and validated using the Wokwi Raspberry Pi Pico simulator with a MicroPython runtime, allowing reliable testing and verification of system performance before real-world implementation.
URL Link	https://wokwi.com/projects/461127750868599809		

SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	GasDetector - sampleInterval : int - gasThreshold : int - status : String + setup() : void + loop() : void MQGasSensor - analogPin : int - digitalPin : int - rawValue : int + readAnalog() : int + isGasDetected() : bool + getPPM() : float + calibrate() : void Buzzer - pin : int - frequency : int + begin() : void + triggerAlarm() : void + beep() : void + stop() : void StepperMotor - steps : int - speed : int + begin() : void + openVent() : void + closeVent() : void + step() : void A4988Driver - stepPin : int - dirPin : int + begin() : void + setSpeed() : void + step() : void + disable() : void uses uses uses uses
COURSE COORDINATOR	Dr ARUNA M G

Details
Abstract : The gas detection system is designed to identify the presence of harmful gases in the environment. It helps in preventing accidents by providing early alerts to users. The system is simple, reliable, and suitable for safety applications in homes and industries.
Use Case: The gas detection system can be used in homes to detect LPG leaks and ensure safety. It is useful in industries and laboratories where harmful gases may be present. The system provides early alerts, allowing quick action to prevent accidents. It helps reduce risks and improves overall safety in different environments.
Introduction: Gas leaks are a serious safety concern that can lead to fires, explosions, and health hazards. A gas detection system helps in identifying the presence of harmful gases at an early stage. This project provides a simple and effective solution to enhance safety. It enables timely alerts, helping users take quick action and prevent accidents.
Objective: • To detect the presence of harmful gases in the environment. • To provide immediate alerts when gas leakage is detected. • To prevent accidents such as fires, explosions, and health hazards. • To develop a simple, reliable, and cost-effective safety system.
General procedure 1. Connect the MQ gas sensor VCC to 5V, GND to GND, and DO (digital output) to a GPIO pin of the Raspberry Pi. 2. Connect the LED and buzzer to GPIO pins with appropriate resistors and common GND. 3. Initialize GPIO pins in the Raspberry Pi using Python (set sensor as input, LED and buzzer as output). 4. Continuously read the digital signal from the gas sensor to detect gas presence.

-
-
5. When gas is detected (HIGH signal), activate the buzzer and LED to alert the user.

Step to Execute:

1. Set up Raspberry Pi Pico environment in Wokwi.
2. Add and connect MQ gas sensor, LED, and buzzer to GPIO pins.
3. Upload the MicroPython code in the editor.
4. Run the simulation and adjust gas sensor values.
5. Observe LED, buzzer, and Serial Monitor output.

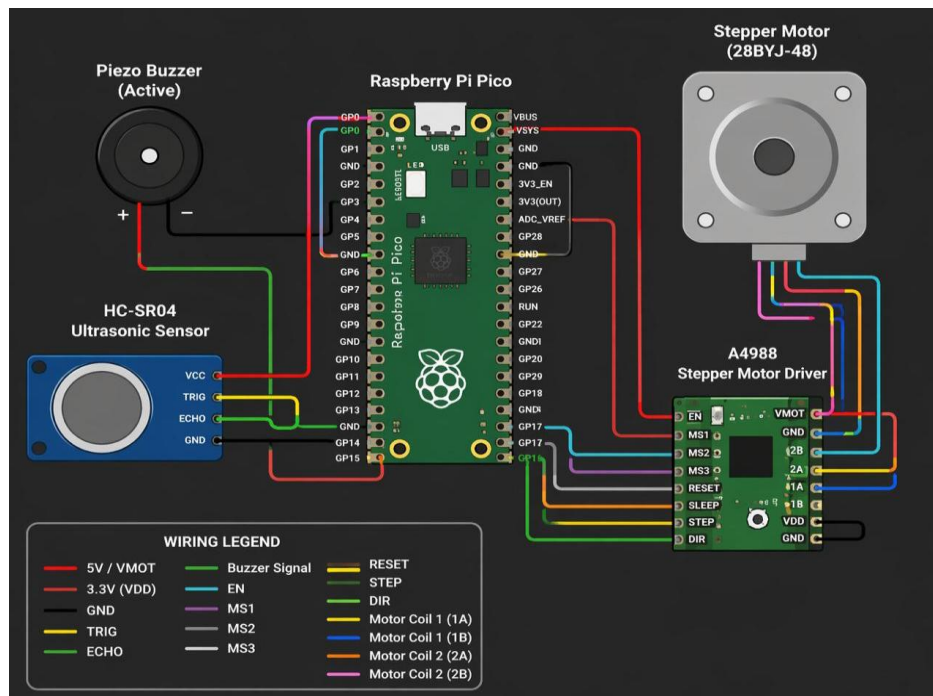
Source Code

```
from machine import Pin, ADC, PWM
import time
gas_sensor = ADC(26)
buzzer = PWM(Pin(15))
step = Pin(2, Pin.OUT)
direction = Pin(3, Pin.OUT)
enable = Pin(4, Pin.OUT)
enable.value(0)
direction.value(0)
threshold = 35000
def rotate_fan():
    step.value(1)
    time.sleep_ms(8)
    step.value(0)
    time.sleep_ms(8)
while True:
    gas_value = gas_sensor.read_u16()
    print("Gas Level:", gas_value)
    if gas_value > threshold:
        buzzer.freq(1000)
        buzzer.duty_u16(30000)
        rotate_fan()
    else:
```

```
buzzer.duty_u16(0)
time.sleep(0.1)
```

Output:

```
System Monitoring...
Gas Level: SAFE
Gas Level: NORMAL
Gas Level: HIGH
GAS LEAK DETECTED!
Activating Buzzer and LED
```

Photos/ Screenshot/Snapshot :**Outcome:**

The system detects the presence of harmful gases in the environment effectively. It provides timely alerts using LED and buzzer when gas levels exceed safe limits. The simulation demonstrates reliable monitoring and quick response to gas leakage. It enhances safety by helping prevent accidents and hazardous situations.

Activity 5 : TinyML simulation in Wokwi

Problem Statement	Sensor systems often produce inaccurate or abnormal readings due to noise, faults, or environmental changes. These irregularities can lead to incorrect decisions if not detected early. Traditional threshold-based methods are not flexible enough to handle complex patterns in sensor data. Therefore, an intelligent system is needed to automatically detect anomalies and ensure reliable monitoring.				
AAT MARKS	Design (5)	Execution (5)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	Wokwi, TinyML, TensorFlow, Potentiometers, Led, Resistors				
MAP SDG	9.2: Promote inclusive and sustainable industrialization. 9.4: Upgrade infrastructure with clean and efficient technologies. 3.9: Reduce deaths and illnesses from hazardous conditions. 11.5: Reduce damage from disasters and hazardous events.				
MAP COURSE OUTCOME of IOT/BFE	CO	Mapping	Justification		
	CO1	Yes	Implementation of an edge-based TinyML system on a microcontroller (RP2040), demonstrating real-time perception and decision-making using sensor data.		
	CO2	Yes	Utilized Python and TensorFlow for model development, followed by TensorFlow Lite conversion for deployment on resource-constrained IoT devices.		
	CO3	Yes	Complete machine learning workflow including data generation, preprocessing, training, evaluation, and model optimization for embedded deployment.		
	CO4	Yes	Integration of hardware components such as sensors (simulated via potentiometers) and actuators (LED), with embedded ML inference on the microcontroller.		
MAP PROGRAM OUTCOME	PO	Mapping	Justification		

	PO1	Yes	Applied knowledge of signal processing, neural networks, and probability-based classification for analyzing sensor data and detecting anomalies.
	PO2	Yes	Conducted machine learning experiments including data generation, training, validation, and evaluation using appropriate performance metrics.
	PO5	Yes	Utilized modern engineering tools such as Google Colab, TensorFlow, TensorFlow Lite, and Wokwi for model development and deployment.
	PO11	Yes	Implemented TinyML, an emerging technology that enables on-device intelligence for real-time and low-power applications.
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO 1	Yes	Demonstrates Python-based machine learning programming for model development, along with C/C++ code generation for deployment on embedded microcontroller platforms, enabling seamless integration of ML with hardware.
	PSO 2	Yes	Applies machine learning techniques to solve the sensor anomaly prediction problem by analyzing patterns in sensor data and classifying conditions into normal, warning, and danger states.
	PSO 3	Yes	Successfully deployed and tested the system using Google Colab for model training and Wokwi ESP32 simulation platform for real-time embedded implementation and validation.
URL Link	https://wokwi.com/projects/461296561453820929		

SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	<pre>classDiagram class AnomalyDetector { -float lastPrint +setup() void +loop() void -printRow(input float[], output float[], mse float, anomaly bool) void } class Autoencoder { -relu(x float) float +runAutoencoder(input float[], output float[]) float } class SensorReader { +int POT_PIN +readSensorChannels(ch float[]) void } class HardwareConfig { +int LED_PIN +int SAMPLE_INTERVAL_MS +analogSetAttenuation() void } class ModelWeights { +float W1[3][2] +float b1[2] +float W2[2][3] +float b2[3] +float ANOMALY_THRESHOLD } AnomalyDetector --> Autoencoder : calls runAutoencoder AnomalyDetector --> SensorReader : calls readSensorChannels AnomalyDetector --> HardwareConfig : configures GPIO Autoencoder --> ModelWeights : reads weights</pre> The diagram illustrates the architecture of an anomaly detection system. It features five main components: AnomalyDetector, Autoencoder, SensorReader, HardwareConfig, and ModelWeights. AnomalyDetector is the central class, managing the overall process. It has a private attribute <code>-float lastPrint</code> and public methods <code>+setup() : void</code>, <code>+loop() : void</code>, and a private method <code>-printRow(input float[], output float[], mse float, anomaly bool) : void</code>. It interacts with Autoencoder (calling <code>runAutoencoder</code>), SensorReader (calling <code>readSensorChannels</code>), and HardwareConfig (configuring GPIO). Autoencoder contains a private method <code>-relu(x float) : float</code> and a public method <code>+runAutoencoder(input float[], output float[]) : float</code>. It depends on ModelWeights (reading weights). SensorReader has a public attribute <code>+int POT_PIN</code> and a public method <code>+readSensorChannels(ch float[]) : void</code>. HardwareConfig has public attributes <code>+int LED_PIN</code> and <code>+int SAMPLE_INTERVAL_MS</code>, and a public method <code>+analogSetAttenuation() : void</code>. ModelWeights contains several public attributes representing model parameters: <code>+float W1[3][2]</code>, <code>+float b1[2]</code>, <code>+float W2[2][3]</code>, <code>+float b2[3]</code>, and <code>+float ANOMALY_THRESHOLD</code>.
COURSE COORDINATOR	Dr ARUNA M G

Details
Abstract : This project presents a TinyML-based sensor monitoring system for detecting anomalies in real-time. Sensor data such as temperature and gas levels are used to train a lightweight neural network model. The trained model is converted to TensorFlow Lite and deployed on a microcontroller for on-device inference. The system classifies conditions into normal, warning, and danger without relying on fixed thresholds. This approach enhances reliability, reduces latency, and enables efficient, low-power intelligent monitoring.
Use Case: This system can be used in smart homes and industries to continuously monitor environmental conditions such as temperature and gas levels. It helps in early detection of hazardous situations like gas leaks or overheating, preventing accidents and equipment damage. The TinyML-based approach enables real-time decision-making directly on edge devices without relying on cloud connectivity. This makes it suitable for low-power, cost-effective, and safety-critical applications.
Introduction: With the increasing use of sensors in smart systems, ensuring the accuracy and reliability of sensor data has become crucial. Traditional monitoring systems rely on fixed thresholds, which are often ineffective in detecting complex or unexpected anomalies. This project introduces a TinyML-based approach for intelligent sensor monitoring and anomaly detection. By leveraging a lightweight neural network model, the system learns patterns in sensor data and classifies conditions in real time. The model is deployed on a microcontroller, enabling on-device inference without the need for cloud connectivity. This results in a fast, energy-efficient, and reliable solution for safety-critical applications.
Objective: • To detect anomalies in sensor data using a TinyML autoencoder. • To compare input and reconstructed values for accurate detection.

- To provide real-time alerts when abnormal patterns are identified.
- To improve reliability and accuracy of sensor-based monitoring systems.

General procedure:

1. Set up the Raspberry Pi Pico environment in Wokwi and add required components (potentiometers and LED).
2. Connect the potentiometers to simulate sensor inputs (temperature and gas) and the LED for anomaly indication.
3. Upload the TinyML model code along with the generated model.h file into the editor.
4. Run the simulation and vary the potentiometer values to represent normal, warning, and anomalous conditions.
5. Observe the LED behavior and Serial Monitor output to analyze anomaly detection results.

Step to Execute:

1. Set up Raspberry Pi Pico simulation in Wokwi.
2. Add components: Raspberry Pi Pico, two potentiometers, and an LED.
3. Connect potentiometers to analog pins (GP26, GP27) and LED to GPIO pin (GP25).
4. Upload the TinyML inference code along with the model.h file.
5. Run the simulation and vary potentiometer values to simulate sensor data.
6. Observe LED indication for normal, warning, and danger conditions.
7. Check Serial Monitor for prediction outputs and anomaly detection status.

Source Code

```
#include <Arduino.h>

#include "model.h"

#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
```

```

#include "tensorflow/lite/schema/schema_generated.h"

#include "tensorflow/lite/version.h"

const tflite::Model*    model;
tflite::MicroInterpreter* interpreter;

TfLiteTensor*          input;
TfLiteTensor*          output;

constexpr int tensor_arena_size = 6 * 1024;
uint8_t tensor_arena[tensor_arena_size];

#define LED_PIN    15
#define TEMP_PIN   26
#define GAS_PIN    27

void setup() {
    Serial.begin(115200);
    while (!Serial) delay(10);
    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);
    model = tflite::GetModel(model_tflite);
    if (model->version() != TFLITE_SCHEMA_VERSION) {
        Serial.println("[ERROR] Model schema version mismatch!");
        while (1);
    }
    static tflite::AllOpsResolver resolver;
    static tflite::MicroInterpreter static_interpreter(
        model, resolver, tensor_arena, tensor_arena_size);
    interpreter = &static_interpreter;
    TfLiteStatus alloc_status = interpreter->AllocateTensors();
    if (alloc_status != kTfLiteOk) {

```

```

    Serial.println("[ERROR] AllocateTensors failed!");
    while (1);
}
input = interpreter->input(0);
output = interpreter->output(0);
Serial.println("[INFO] Model loaded. Starting inference loop...");
}
void loop() {
    int raw_temp = analogRead(TEMP_PIN);
    int raw_gas = analogRead(GAS_PIN);
    float temp = raw_temp / 1023.0f;
    float gas = raw_gas / 1023.0f;
    input->data.f[0] = temp;
    input->data.f[1] = gas;
    TfLiteStatus invoke_status = interpreter->Invoke();
    if (invoke_status != kTfLiteOk) {
        Serial.println("[ERROR] Invoke failed!");
        return;
    }
    float normal = output->data.f[0];
    float warning = output->data.f[1];
    float danger = output->data.f[2];
    Serial.println("===== Prediction =====");
    Serial.print(" Raw → Temp: "); Serial.print(raw_temp);
    Serial.print(" Gas: ");      Serial.println(raw_gas);
    Serial.print(" Norm → Temp: "); Serial.print(temp, 3);
    Serial.print(" Gas: ");      Serial.println(gas, 3);
}

```

```

Serial.print(" Normal : "); Serial.println(normal, 4);
Serial.print(" Warning : "); Serial.println(warning, 4);
Serial.print(" Danger : "); Serial.println(danger, 4);
if (danger > 0.6f) {
    digitalWrite(LED_PIN, HIGH);    // LED ON → DANGER
    Serial.println(" >> DANGER DETECTED! <<");
} else if (warning > 0.5f) {
    digitalWrite(LED_PIN, LOW);
    Serial.println(" >> WARNING LEVEL");
}
else {
    digitalWrite(LED_PIN, LOW);
    Serial.println(" >> NORMAL");
}
Serial.println("=====");
delay(1000);
}

```

ML Source Code:

```

import tensorflow as tf
import numpy as np
X = np.array([[0.1],[0.2],[0.15],[0.9],[1.0],[0.85]])
y = np.array([0,0,0,1,1,1])
model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation='relu', input_shape=(1,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

```

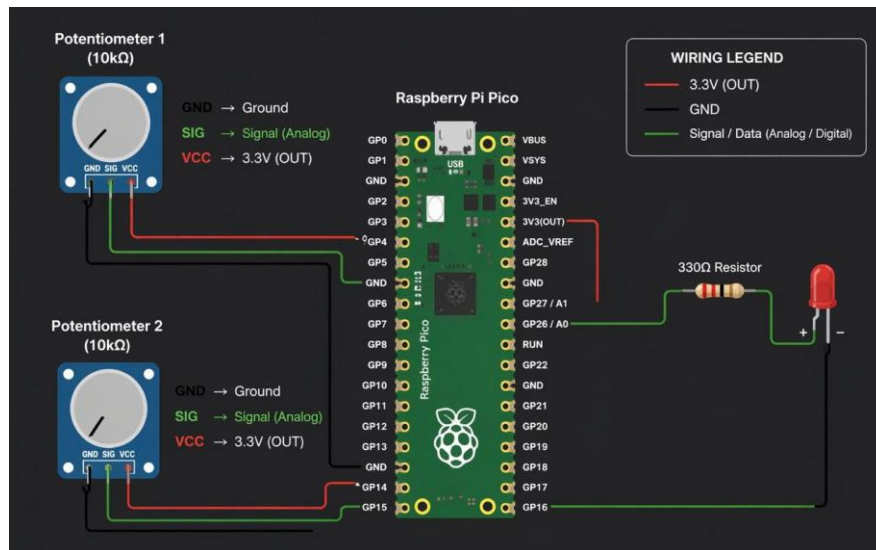
```
model.fit(X, y, epochs=100)
```

```
model.save("model.h5")
```

Output:

```
=====
===== Prediction =====
Temp: 1.000 Gas: 0.867
Normal : 0.4051
Warning : 0.1854
Danger : 0.4095
>> NORMAL
```

Photos/ Screenshot/Snapshot :



Outcome:

The system successfully detects anomalies in sensor data using a TinyML model deployed on a microcontroller. It accurately classifies inputs into normal, warning, and danger conditions in real time. The LED provides immediate visual indication of critical situations, enhancing

system responsiveness. The use of on-device inference ensures low latency and energy-efficient operation without cloud dependency. Overall, the project demonstrates a reliable and scalable approach for intelligent sensor monitoring.

Dr. Aruna M G

Course Coordinator

Dr. Vindhya P Malagi

Head of the Department

Artificial Intelligence & Machine Learning

Dayananda Sagar College of Engineering

Bangalore-560111